

Segmentation Engine Documentation

September 8, 2025

1 Scope

This document describes the Segmentation Engine, a C++ HTTP service for route analysis and segment extraction. It has been prepared in accordance with ISO/IEC/IEEE 26514:2008, which defines structure and content for software user documentation.

2 Normative References

- ISO/IEC/IEEE 26514:2008 – Systems and software engineering – Requirements for designers and developers of user documentation.
- ISO 8601:2019 – Date and time representation.

3 Terms and Definitions

Segment A contiguous portion of a route, represented as GeoJSON geometry and associated metadata.

Route signal A sampled representation of map variables (elevation, curvature, ...) indexed by distance along the route.

Bounding box Geographic region defined by west, south, east and north coordinates.

4 System Overview

The engine exposes a RESTful API that receives map data, builds route signals, performs wavelet based segmentation and persists results to a MySQL database. The major modules are:

- **core**: algorithmic components such as the route signal builder, segmentation logic and wavelet utilities.
- **http**: request routing and REST endpoint handlers built on `cpp-httplib`.
- **infra**: database adapter for MySQL persistence.
- **models**: C++ types representing OSRM responses, segments and configuration parameters.

5 Installation and Operation

5.1 Prerequisites

CMake 3.15 or newer, a C++17 capable compiler, Docker and a running MySQL server.

5.2 Build and Deployment

The provided script builds and runs the engine inside a Docker container:

```
./build_and_deploy.sh
```

It performs three steps:

1. Build an image tagged `segmentation_engine:latest`.
2. Remove any existing container named `segmentation_container`.
3. Launch a new container on port 5005 with MySQL parameters (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASS`, `DB_NAME`) supplied via the environment.

Internally, the script issues the following Docker commands:

```
docker build -t segmentation_engine:latest .
docker rm -f segmentation_container >/dev/null 2>&1 || true
docker run -d --name segmentation_container --network host \
  -e DB_HOST=127.0.0.1 -e DB_PORT=3306 -e DB_USER=routeseg_user \
  -e DB_PASS=changeme-user -e DB_NAME=routeseg -p 5005:5005 \
  segmentation_engine:latest
```

Container logs can be monitored via:

```
docker logs -f segmentation_container
```

For a manual build without Docker:

```
cmake -S . -B build
cmake --build build
./build/segmentation_engine
```

5.3 Configuration

Runtime settings, including the HTTP port and enabled endpoints, are stored in `config/settings.json`. Database credentials are compiled into `src/main.cpp` and should be configured prior to deployment.

5.4 Execution

After deployment the service is reachable on `http://localhost:5005` and mounts the `public` directory under `/static`. POST and GET routes are registered from the configuration file.

To consolidate endpoints and avoid Cross-Origin Resource Sharing (CORS) issues, the parent `segmentation` directory provides a lightweight Nginx proxy. From that directory run:

```
./proxy.sh
```

The script starts a container named `quick-proxy` that forwards port 8081 to the Segmentation Engine at 5005, exposing the service at `http://localhost:8081`. Additional proxy ports 8082 and 8083 target the viewer and database UI, allowing browser clients to interact with all components through a single origin.

6 REST API

Table 1 lists the available endpoints. JSON bodies use UTF-8 encoding and timestamps follow ISO 8601.

Table 1: REST endpoints

Method/Path	Description
POST /upload	Accepts a map JSON file and stores it under <code>uploads/</code> . Returns the storage path.
GET /view?map= <i>path</i>	Renders an interactive HTML viewer comparing raw OSRM geometry to the derived route signal.
GET /viewLab?map= <i>path</i>	Redirects to the Signal Lab user interface.
GET /lab/meta?map= <i>path</i>	Lists available uploads and optionally returns metadata for a specific map.
POST /lab/resample	Builds a route signal and resamples requested variables onto a uniform distance grid.
POST /wavelet	Performs wavelet-based terrain segmentation and optionally persists segments to the database.
GET /segments	Retrieves previously persisted segments intersecting a geographic bounding box.

7 Algorithms

Segmentation is accomplished by constructing a uniformly sampled route signal from the uploaded map and applying a wavelet-based energy detector. High energy regions correspond to changes in elevation or curvature and are converted into segments. Utilities in `core/wavelets` implement the FFT based rolling energy calculation. Existing segments can be matched against new routes to avoid redundant processing.

8 Data Management

Segment metadata is stored in a MySQL schema defined by `schema.sql`. The `infra::MySQLSegmentDB` component manages connections, inserts new segments and queries by bounding box.

9 Error Handling and Logging

Fatal signals (segmentation faults, aborts, arithmetic errors) are trapped and a stack trace is printed to the server log. Endpoint handlers translate exceptions into HTTP 500 responses containing a diagnostic message.

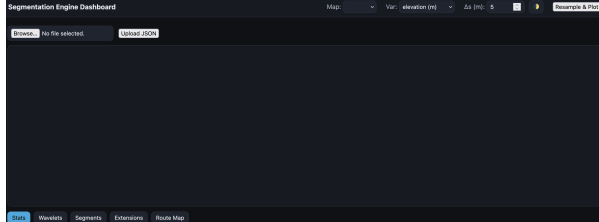
10 Conformance

The Segmentation Engine and this document use ISO 8601 timestamps and the terminology defined in ISO/IEC/IEEE 26514:2008. All numeric values are expressed using SI units.

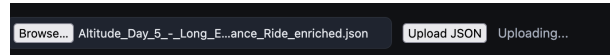
11 Application Screenshots

11.1 Dashboard and Data Upload

In line with ISO/IEC/IEEE 26514:2008 guidance on task-oriented documentation, this sequence demonstrates the primary user workflow: initial dashboard state, file transfer progression, confirmation of successful ingestion, and verification via route–OSRM alignment. The figure substantiates that the ingest–validate loop is operational end to end.



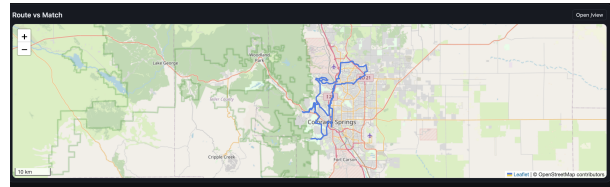
(a) Default dashboard on initial load.



(b) Upload in progress.



(c) Upload complete.



(d) Route vs matched geometry.

Figure 1: Loading, uploading, and path matching workflow.

11.2 Database Viewer

This view evidences persistence and retrieval capabilities as required by the standard’s “data management and storage” descriptions. It shows that extracted segments are indexed, queryable, and auditable, supporting reproducibility of results and post hoc analysis.

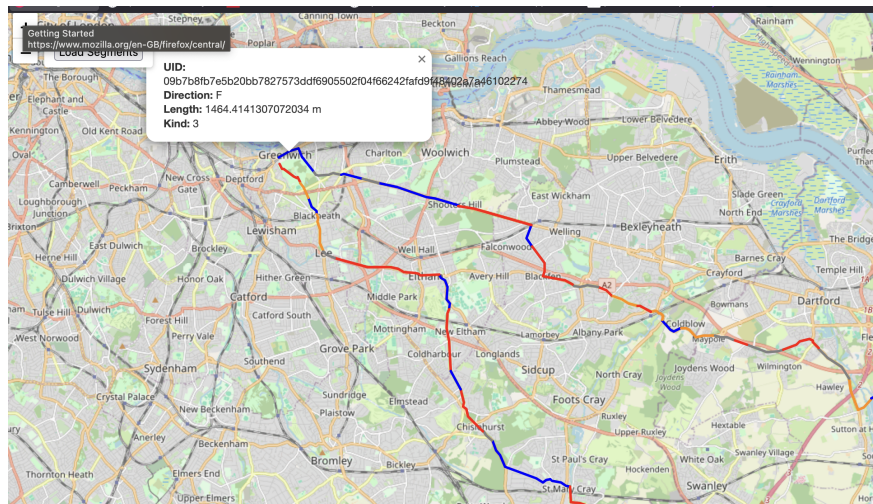
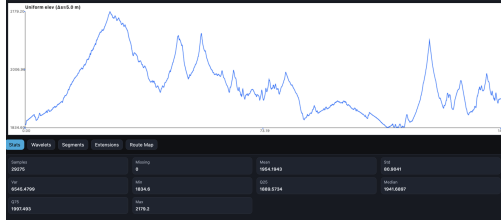


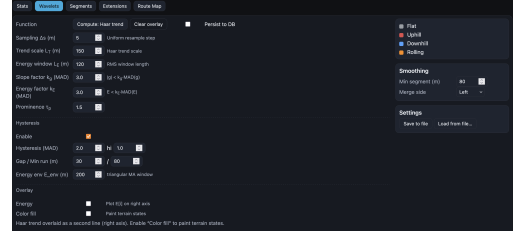
Figure 2: Interactive segment database inspector.

11.3 Signal and Wavelet Panels

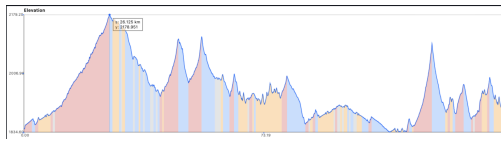
These panels expose algorithm configuration and outputs, satisfying the standard's expectation for clarity of computational steps. Elevation-versus-distance, tunable wavelet parameters, and the scalogram together justify threshold and segmentation choices, enabling readers to trace input settings to resultant segments.



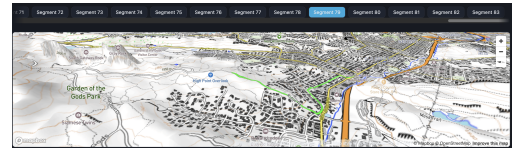
(a) Elevation view ($\Delta s = 5\text{m}$).



(b) Wavelet controls.



(c) Scalogram with trend overlay.



(d) Segmentation options.

Figure 3: Signal plots, wavelet parameters, and segmentation settings.

12 Extensions Graph

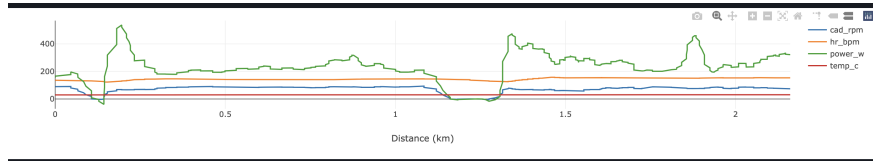
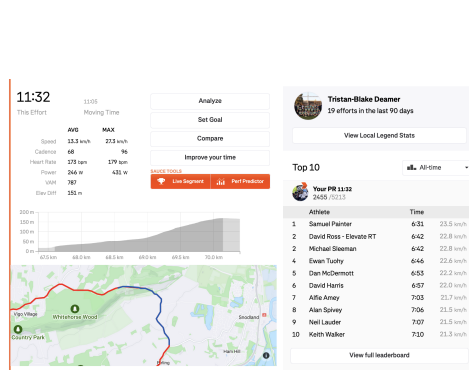
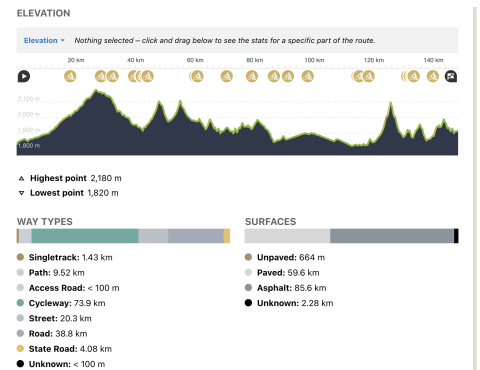


Figure 4: Auxiliary signals (extensions) aligned to distance.

12.1 External Platform Views



(a) Strava route view.



(b) Komoot route view.

Figure 5: Comparison: Strava vs Komoot.